



Weather Pattern Classification Using Big Data Technologies

Group : Cloud Chasers

Savitha Sathyan

Anagha Sathish

Master's in IT, Halmstad University

September 12, 2025

ABSTRACT

This project evaluates the impact of using Apache Spark for parallelizing Random Forest classification on weather datasets of varying sizes and granularities. Two datasets were selected: a small-to-medium daily weather dataset and a large hourly weather dataset, both containing multiple meteorological features. Baseline experiments were conducted using scikit-learn on sampled subsets, while Spark MLlib was employed for distributed training on the full datasets. The results show a significant improvement in classification accuracy with the larger hourly dataset, achieving over 92% accuracy using Spark and over 99% accuracy on sampled data with scikit-learn. Training time scaled accordingly with data size and complexity, highlighting the trade-offs between model accuracy and computational cost. This analysis provides insights into when and how Spark-based parallelization is beneficial for large-scale classification problems in meteorological data.

CONTENTS

1	INTRODUCTION	1
2	DATASET	2
2.1	Daily Dataset	2
2.2	Hourly Dataset	2
2.3	Metadata and Units Information	2
2.4	Dataset Summary Table	3
3	METHODOLOGY	4
3.1	Data Preprocessing	4
3.2	Random Forest Classifier	5
3.3	Experimental Setup	5
3.4	Computing Environment	5
3.5	Baseline Experiments	6
3.6	Spark Method	6
3.7	Evaluation Metrics	6
3.8	Limitations	6
4	EXPERIMENTS AND RESULTS	7
4.1	Parallelization on the Dataset	7
4.2	Baseline Methods on Datasets	8
5	ANALYSIS AND EXPERIMENTATION	10
5.1	Limitations	11
6	CONCLUSION	12

INTRODUCTION

In recent years, the growth of big data has transformed the way we approach data analysis and machine learning. Parallel computing frameworks like Apache Spark enable efficient processing and modeling of large-scale datasets, which would otherwise be infeasible on single machines. This project investigates the effectiveness of using Sparks distributed computing capabilities to perform classification on weather datasets with different sizes and time resolutions.

We selected two datasets for this study : a small-to-medium daily weather dataset and a large hourly weather dataset. Both contain rich meteorological features such as temperature, humidity, precipitation, and wind characteristics. Our goal is to understand how Sparks parallelization impacts classification performance and resource usage, compared to traditional baseline approaches running on sampled data subsets locally.

Random Forest, a widely-used ensemble learning method, was chosen as the classification algorithm due to its robustness and ability to handle high-dimensional data. We evaluated model accuracy and training/prediction times across different configurations, providing insight into the trade-offs involved in scaling machine learning workflows using big data technologies.

DATASET

This project utilizes two weather datasets stored as Parquet files, chosen to represent different sizes and temporal granularities for classification tasks. Additionally, CSV files containing metadata and units information were used to better understand the datasets structure and feature meanings.

2.1 DAILY DATASET

The daily dataset contains approximately 5.25 million records of aggregated daily weather observations collected from multiple cities, starting from the year 1950. It includes features such as maximum, minimum, and mean temperatures, apparent temperature, daylight and sunshine durations, precipitation sums and hours, wind speeds and gusts, wind direction, radiation measures.

The target variable is the weather condition code: weather code representing different weather classes. This dataset is used to evaluate classification performance on a small to medium scale, providing a baseline for understanding how well machine learning models can classify weather conditions using daily aggregated data.

2.2 HOURLY DATASET

The hourly dataset consists of roughly 154 million rows of detailed hourly weather data gathered from various locations since 1950. It includes meteorological attributes such as temperature, relative humidity, dew point, apparent temperature, precipitation, atmospheric pressures, cloud cover at multiple layers, vapor pressure deficit, wind speeds and directions at various heights, soil temperature and moisture at different depths, and radiation measurements.

The classification target variable is also weathercode. This dataset is intended to assess the benefits and challenges of applying big data parallel processing techniques on a large-scale, high-resolution dataset. It allows analysis of how increased temporal detail and data volume impact classification accuracy and computational efficiency.

2.3 METADATA AND UNITS INFORMATION

To complement the Parquet datasets, several CSV files were provided containing city information and detailed units descriptions for the daily and hourly weather features. These files helped clarify the mean-

ing, scale, and units of each attribute, enabling more informed feature selection and preprocessing decisions during the experimental phase.

2.4 DATASET SUMMARY TABLE

Dataset	Size (Rows)	Time Res.	Features	Description	Target
Daily	~5.25M	Daily	16	Aggregated daily weather data from cities	weather_code
Hourly	~154M	Hourly	11	Detailed hourly meteorological data	weather_code

Table 1: Summary of weather datasets used in the project

METHODOLOGY

This project employed a comparative experimental framework to evaluate the performance of Random Forest classification on two weather datasets differing in size and temporal resolution.

3.1 DATA PREPROCESSING

Before model training, both the daily and hourly datasets underwent a structured preprocessing workflow to ensure data consistency, feature readiness, and suitability for machine learning tasks.

For both datasets, the initial step was to select relevant features while excluding non-numeric columns such as city names, timestamps, and other descriptive attributes. Only numerical meteorological variables and the target variable weather code were retained. The rationale behind this selection was twofold:

1. Non-numeric fields do not directly contribute to weather classification in their raw form and would require additional feature engineering to be meaningful.
2. Numerical meteorological variables directly capture atmospheric conditions such as temperature, humidity, pressure, wind, and radiation which are physically and statistically linked to weather states. These continuous variables provide predictive power for classification models, making them the most relevant features to include in the baseline approach.

To handle missing values, a row-wise drop strategy was applied using the `dropna()` function. This ensured that only complete rows, without null values in the selected features or the label column, were considered for downstream processing.

Since the target variable weather code is categorical but stored as numerical codes, it was processed using `StringIndexer` from PySpark ML. This step encoded the weather codes into integer labels suitable for classification tasks, while preserving the mapping between original weather codes and indexed labels.

Next, all selected numerical features were combined into a single feature vector using `Vector Assembler`. This transformation generated a new column called `features`, consolidating multiple meteorological variables into a format compatible with Spark ML algorithms.

Finally, the preprocessed dataset was split into training and test subsets using an 80/20 ratio with a fixed random seed for reproducibility. To optimize training performance, the training partition

was cached in memory, reducing repeated computation during model fitting.

Through this preprocessing pipeline, both the daily and hourly datasets were transformed from raw tabular data into structured feature-label pairs, enabling scalable machine learning workflows in Spark.

3.2 RANDOM FOREST CLASSIFIER

Random Forest is an ensemble learning method that builds multiple decision trees and aggregates their outputs to improve classification accuracy and reduce overfitting. It is effective in handling high-dimensional data and capturing complex relationships without extensive hyperparameter tuning. The method robustness and interpretability make it well-suited for classification problems with heterogeneous meteorological features. Additionally, Random Forest implementations in both Spark MLlib and scikit-learn enabled consistent experimentation across distributed and local computing environments.

3.3 EXPERIMENTAL SETUP

For the distributed experiments, the entire datasets were split into training (80%) and testing (20%) sets using Spark random Split function with a fixed random seed for reproducibility. Training datasets were cached in memory to optimize processing speed during model fitting.

Random Forest classifiers were trained independently on the daily and hourly datasets using Spark MLlib. A number of 10 trees was selected for the Random Forest as a balance between model complexity and computational cost, following preliminary experiments. Training and prediction times were recorded to assess computational efficiency.

3.4 COMPUTING ENVIRONMENT

All experiments were conducted in Google Colab, leveraging a cloud-based environment equipped with Apache Spark (version 3.5.0), Python (version 3.8), and necessary ML libraries. The runtime environment (Google Colab) provided approximately 12 GB of RAM and 2 virtual CPU cores, facilitating distributed processing while remaining accessible and convenient.

3.5 BASELINE EXPERIMENTS

In the scikit-learn preprocessing workflow, the weather datasets were loaded and restricted to a curated set of numeric features that are most predictive for classification. The target variable was converted into numeric form using `LabelEncoder`, ensuring it could be interpreted by scikit-learn models. Missing values were handled through row removal to maintain data quality, and the selected features were normalized and assembled into a model-ready matrix using a `ColumnTransformer` with `StandardScaler`. This produced a clean, consistent, and comparable input representation that could be directly used for baseline classification experiments in scikit-learn.

3.6 SPARK METHOD

In the Spark-based preprocessing pipeline, the weather datasets were first ingested from Parquet files and narrowed to a set of relevant numeric features to reduce noise and dimensionality. The target variable (weather code) was transformed into a numeric label using `StringIndexer`, ensuring compatibility with Spark ML classifiers. Any rows containing missing values were dropped to maintain dataset integrity. Finally, all selected predictors were consolidated into a single vector representation using `VectorAssembler`, creating a structured and efficient feature set for downstream model training and evaluation within Spark distributed framework.

3.7 EVALUATION METRICS

Model performance was evaluated using classification accuracy and weighted F1 score, capturing overall correctness and balance between precision and recall across weather classes. Training and prediction times were also recorded to assess computational resource demands.

3.8 LIMITATIONS

The experiments were limited by the resources available within Google Colab, including memory constraints and CPU availability, which influenced the choice of model complexity and dataset sampling size. Additionally, Spark cluster configuration was constrained to a single-node environment, limiting the scalability demonstration of distributed processing.

EXPERIMENTS AND RESULTS

This section presents the results of Random Forest classification experiments conducted on the daily and hourly weather datasets using both Apache Spark MLlib and scikit-learn. The primary metrics considered are training time, prediction time, classification accuracy, and weighted F1 score.

4.1 PARALLELIZATION ON THE DATASET

Daily Dataset:

In this experiment, we evaluated the effect of Spark parallelism on the daily dataset by varying the number of partitions (2, 4, 8, and 32) and measuring training time, prediction time, and test accuracy. With 2 partitions, the model required 171.28 seconds for training and 0.16 seconds for prediction, achieving an accuracy of 0.6335. Increasing to 4 partitions reduced the training time to 108.52 seconds and slightly lowered the prediction time to 0.15 seconds, while accuracy remained constant at 0.6335. When the dataset was split into 8 partitions, training became even faster at 91.83 seconds and prediction time dropped to 0.14 seconds, with accuracy improving marginally to 0.6382. At 32 partitions, training time slightly increased again to 95.92 seconds and prediction time rose to 0.22 seconds, while accuracy stayed at 0.6382. These results show that parallelism significantly reduces training time up to an optimal point, with 8 partitions providing the best balance between speed and accuracy. Beyond this point, such as at 32 partitions, the overhead of managing too many partitions begins to offset the benefits of parallelism. Importantly, the accuracy of the model remained consistent across all configurations, demonstrating that changes in partitioning mainly influenced computational efficiency rather than predictive performance. A detailed summary of these results is presented in Table 2, which clearly shows the relationship between partitions, training time, prediction time, and test accuracy.

Partitions	Training Time (s)	Prediction Time (s)	Test Accuracy
2	171.28	0.16	0.6335
4	108.52	0.15	0.6335
8	91.83	0.14	0.6382
32	95.92	0.22	0.6382

Table 2: Results Table showing training time, prediction time, and accuracy across different partitions.

Hourly Dataset:

To analyze the effect of partitioning on the hourly dataset, experiments were conducted using 2 and 8 partitions. For each configuration, the training time, prediction time, and test accuracy were measured. With 2 partitions, the model required 4125.04 seconds for training and 0.12 seconds for prediction, achieving an accuracy of 0.9352. When the dataset was split into 8 partitions, training time decreased to 3311.59 seconds and prediction time was reduced to 0.06 seconds, while accuracy slightly improved to 0.9433. These results highlight the benefit of increasing partitions for large datasets: parallelism reduced computation time while maintaining and even improving accuracy. Compared to the daily dataset experiments, the hourly dataset was much larger and more computationally intensive, which explains the longer training times overall. However, the consistent improvement with higher partitions demonstrates that Spark parallelism is particularly advantageous for handling such large-scale data. A detailed summary of these results is shown in Table 3 below.

Partitions	Training Time (s)	Prediction Time (s)	Test Accuracy
2	4125.04	0.12	0.9352
8	3311.59	0.06	0.9433
32	4793.95	0.18	0.9235

Table 3: Hourly Dataset Performance with Different Partitions

4.2 BASELINE METHODS ON DATASETS

Daily Dataset:

To establish a reference point, a baseline classification experiment was conducted on the full daily dataset using scikit-learn. The training time for this experiment was 248.74 seconds, while prediction required 7.19 seconds. The model achieved a test accuracy of 0.7148, which serves as a benchmark for evaluating distributed approaches. Compared to the Spark experiments, the baseline provided relatively

efficient training and strong accuracy, although prediction was slower due to scikit-learn sequential processing. This baseline result is important as it highlights the trade-offs between single-machine learning and distributed processing, providing a clear point of comparison for the subsequent Spark-based experiments.

Hourly Dataset:

To overcome memory crashes in Colab when attempting to load the full hourly dataset at once, a year-by-year slicing approach was implemented using PyArrow predicate pushdown. This allowed the model to train on manageable yearly subsets and aggregate the results. For the years 1950â1954, approximately 10.25 million rows were processed in total. The baseline Random Forest classifier achieved very high accuracy across all years, averaging 0.9970, with average training time of 40.34 seconds per year and prediction time of 0.59 seconds. The results show that, while the dataset is extremely large and cannot be processed in its entirety on Colab due to RAM and compute constraints, careful year-by-year sampling enabled successful baseline experimentation. Accuracy remained consistently high across all years (between 0.9965 and 0.9974), demonstrating the strength of the model on this dataset. However, this approach required compromises in scalability, as only limited slices could be tested at a time rather than the full dataset in one run.

The experimentation phase involved running classification tasks on two datasets, the daily dataset and the hourly dataset, using both baseline (scikit-learn) methods and distributed Spark implementations with varying degrees of parallelism. The goal was to measure and compare training time, prediction time, and test accuracy across different configurations.

For the daily dataset, the baseline model using scikit-learn achieved an accuracy of 0.7148, with a training time of 248.74 seconds and a prediction time of 7.19 seconds. In contrast, Spark experiments with 2, 4, 8, and 32 partitions showed slightly lower accuracy values in the range of 0.6335 to 0.6382. However, Spark demonstrated significant differences in training efficiency as partitions increased. Training time dropped from 171.28 seconds with 2 partitions to 91.83 seconds with 8 partitions, before slightly increasing again with 32 partitions due to overhead. Prediction times with Spark were considerably faster than the baseline, staying under 0.25 seconds in all cases. These results suggest that Spark parallelism improves scalability and prediction efficiency but may sacrifice some accuracy compared to the baseline implementation.

For the hourly dataset, the scale of the data introduced additional challenges. Running the entire dataset at once in Colab was not feasible, as it repeatedly led to memory crashes and session timeouts. To address this, a year-by-year slicing approach was used with PyArrow to load only manageable chunks. Using this method, baseline experiments were conducted successfully for five consecutive years (1950-1954), covering more than 10.25 million rows. The results were remarkably consistent, with average accuracy of 0.9970, average training time of 40.34 seconds, and prediction time of 0.59 seconds. This demonstrated that, although Colab could not handle the full dataset at once, baseline scikit-learn methods remained highly effective when applied in smaller slices.

In contrast, Spark was able to process the full hourly dataset without slicing, making it the more scalable option. With 2 partitions, Spark required 4125.04 seconds for training and achieved 0.9352 accuracy. Increasing to 8 partitions reduced the training time to 3311.59 seconds and improved accuracy slightly to 0.9433, while prediction time dropped from 0.12 seconds to 0.06 seconds. Compared to the baseline slicing method, Spark was slower in training but offered the advantage of handling the entire dataset in a single run, ensuring a

more comprehensive model without the need to merge yearly subsets.

Overall, the experiments highlight important trade-offs. The scikit-learn baseline was highly effective on both the daily dataset and yearly slices of the hourly dataset, delivering strong accuracy and relatively short training times. However, its scalability was constrained by Colab hardware limits, which prevented full-dataset processing. Spark, on the other hand, offered clear advantages in scalability and prediction speed, particularly for the hourly dataset, though it showed modestly lower accuracy than the baseline on the daily dataset. These findings suggest that the choice between baseline methods and distributed frameworks depends strongly on dataset size, available compute resources, and the balance required between accuracy, efficiency, and scalability.

5.1 LIMITATIONS

During the course of this project, several limitations were encountered that influenced the scope and outcomes of experimentation. A key challenge was the computational constraint of Google Colab, where the hourly dataset proved too large to be processed using baseline scikit-learn methods, often leading to crashes due to memory exhaustion and session timeouts. While Spark helped overcome these issues through distributed processing, the execution time for larger partitions was still significant, limiting the number of repeated trials and hyperparameter tuning experiments that could be performed. Another limitation was the trade-off observed between accuracy and scalability: while Spark offered faster training and prediction for large datasets, it produced slightly lower accuracy on the daily dataset compared to the baseline. Additionally, restrictions on system resources meant that experiments were constrained to a limited range of partition sizes, and advanced techniques such as model optimization or deeper hyperparameter exploration could not be fully realized. These limitations suggest that more powerful compute infrastructure would allow for broader experimentation and potentially improved model performance.

CONCLUSION

This project demonstrated the effectiveness of both baseline and distributed approaches for weather data classification across daily and hourly datasets. The baseline scikit-learn experiment on the daily dataset provided a strong benchmark with an accuracy of 0.7148, highlighting the advantages of single-machine methods for smaller datasets in terms of accuracy and relatively short training times. However, its longer prediction time and inability to handle the much larger hourly dataset underscored the scalability limitations of this approach. Spark-based experiments addressed these challenges by enabling distributed training and prediction, achieving faster computation and making it possible to process the hourly dataset, which was otherwise unmanageable in Colab. The results showed that Spark performed especially well with larger datasets, reducing training and prediction times while maintaining competitive accuracy levels, although it delivered slightly lower accuracy on the smaller daily dataset compared to the baseline.

Overall, the experiments highlight a clear trade-off: while traditional machine learning methods excel in accuracy and efficiency for smaller datasets, distributed frameworks such as Spark are essential for scaling to larger data volumes where single-machine methods fail. This reinforces the importance of selecting the right computational approach based on dataset size and project requirements.